

Introduction to Security

- Confidentiality: Prevention of unauthorised disclosure of information
- Integrity: Prevention of unauthorised modification of information of processes
- Availability: Prevention of unauthorised withholding of information of resources
- Thread Model: Describes security of a system by the class of attacks it can prevent
- A class of attacks is described by the attacker's goals and capabilities
- Can be used to compare security of systems
- Ease of Use: Security mechanisms interfere with working patterns users are originally familiar with
- Performance: Security mechanisms consume more computing resources
- Cost: Security mechanisms are expensive to develop and manage
- CVE: Common Vulnerabilities and Exposures, public list of entries
- Zero Day: Vulnerabilities that are discovered but not published, no reaction time
- Adversarial thinking: Thinking like attacker when designing secure systems
- Threat: Set of circumstances that has the potential to cause loss or harm
- Vulnerability: A weakness in the system
- Control: A means to counter threats
- A threat is blocked by control of a vulnerability

Encryption

Definitions

- Symmetric Key Encryption Scheme: Consists of encryption and decryption algorithm
- Correctness: $D_k(E_k(x)) = x$
- Security (Informally): Eavesdropper cannot derive useful information about key or plaintext even if they can probe
- Cryptography: Study of techniques in securing communication in the presence of attackers who have access to it

Thread Model

- Attacker's Goals:
 - Total Break: Attacker wants to find the key
 - Partial Break: Many definitions, for e.g. attacker wants to decrypt a specific ciphertext
 - Distinguishability: Attacker can distinguish ciphertexts between one plaintext and another
 - An attacker who can achieve total break can also achieve partial break and distinguishability
- Attacker's capability:
 - Ciphertext only: Attacker has a collection of ciphertexts, and may know some properties of it
 - Known plaintext: Attacker has a collection of plaintexts and corresponding ciphertexts
 - Chosen plaintext attack (CPA): Attacker has access to encryption oracle, which they can feed any plaintext to to get the corresponding ciphertext
 - Chosen ciphertext attack (CCA2): Same as CPA but attacker can choose ciphertext to use with decryption oracle to get plaintext
- If a cipher can defend against a decryption oracle, then it can defend against all other weaker forms
- If a cipher is secure against CCA2, it is also secure against CPA

Classical Ciphers

Substitution Cipher

- The plaintext and ciphertext are strings over a set of symbols
- The key is a substitution table which is a bijective function from the set of symbols to itself
- The key space is the set of all possible keys with the size being the total number of possible keys
- The key size is the number of bits required to represent a key
- Encryption: Given plaintext $X = x_1x_2...x_n$ and key S , output ciphertext $E_S(X) = S(x_1)S(x_2)...S(x_n)$
- Decryption: Given ciphertext $C = c_1c_2...c_n$ and key S , output plaintext $D_S(C) = S^{-1}(c_1)S^{-1}(c_2)...S^{-1}(c_n)$
- Simple attack is to perform an exhaustive search on the key space
- To be secure, exhaustive search must be computationally infeasible
- Running time depends on the size of the key space, needing $n!$ loops where n is the size of the alphabet
- Known plaintext attack: Attacker has a plaintext and ciphertext, and can use it to reverse engineer some entries in the key
- Example would be headers in packets where starting is fixed
- Ciphertext only attack: Attacker knows plaintext is an English sentence, so they brute force all keys on the ciphertext, only considering when the result has English words
- Frequency analysis: Attacker can guess plaintext by mapping letters to their most frequently occurring counterparts
- Substitution cipher is not secure under both types of attacks

Permutation Cipher

- Group plaintext into blocks of t characters then apply a permutation function to each block, where the permutation is a bijective function from $1...t$ to itself
- $p = (p_1, p_2, ..., p_t)$ which shifts the character i to position p_i
- Fails miserably under known-plaintext attack
- Performing substitution or substitution twice does not increase difficulty of attack, but interlacing them can make it more difficult

One Time Pad

- Encryption: Given n bit plaintext $x_1, ..., x_n$ and n bit key $k_1, ..., k_n$, $C = (x_1 \oplus k_1), ..., (x_n \oplus k_n)$
- Decryption: Given n bit ciphertext $c_1, ..., c_n$ and n bit key $k_1, ..., k_n$, $X = (c_1 \oplus k_1), ..., (c_n \oplus k_n)$
- Key cannot be reused, but must be same size as plaintext
- Correctness is proven by $(x \oplus k) \oplus k = x \oplus (k \oplus k) = (x \oplus 0) = x$
- Attacker can derive key with a plaintext and ciphertext pair, but it is useless since keys are only used once
- Exhaustive search does not work, and one time pads do not leak any information of plaintext

Modern Ciphers

DES / Exhaustive Search

- Quantify security of encryption scheme by length of key, a scheme with larger keys is more secure wrt exhaustive search
- DES uses 56 bit keys, but exhaustive search is now feasible due to improvements in computing power

AES (Advanced Encryption Standard)

- Block length of 128 while key length can be 128, 192, or 256 bits
- Currently no known attacks

Block Cipher & Mode-of-Operations

- DES and AES are block ciphers which are designed for fixed size inputs and outputs
- Block cipher is applied to each block in a large plaintext
- Mode-of-operation specifies how encryption is extended from a single block to multiple blocks
- ECB (Electronic Code Book) mode: Plaintext is divided into blocks before block cipher is applied to each block all with the same key
- ECB leaks information, and is deterministic, so additional mechanisms are required to prevent leakage
- CBC (Cipher Block Chaining): Initialisation Vector arbitrarily chosen
- Encryption: $c_0 = IV, c_i = E_k(x_i \oplus c_{i-1}), i > 0$ is used to generate ciphertext y
- Decryption: $P_1 = D_k(C_1) \oplus IV, P_i = D_k(C_i) \oplus C_{i-1}, i > 0$
- CTR (Counter Mode): Each block x_i uses $IV + i$, stream cipher
- Encryption: $c_i = x_i \oplus E_k(IV + i)$
- Decryption: $p_i = c_i \oplus E_k(IV + i)$
- GCM (Galois Counter Mode): Authenticated Encryption, produces an extra tag for authentication, secure in presence of decryption oracle

Stream Cipher and IVs

- Stream cipher has an Initialisation Vector (IV)
- Pseudorandom sequence is generated from secret key and IV
- Final ciphertext contains IV followed by output of OTP encryption
- For decryption, IV is extracted from ciphertext, so that same pseudorandom sequence can be generated and used to obtain plaintext
- IV should be changed to prevent attacker from eavesdropping and decoding
- If IVs are different in two different instances of encryption, the two pseudorandom sequences will also be different, and xoring them will not cancel them out
- CBC mode also needs IV for non-determinism

Attacks

Triple DES & MITM

- DES is not secure wrt computing power, can be improved using multiple encryptions
- Meet in the Middle: Attacker brute forces encryption from plaintext with first key and decryption from ciphertext with second key
- Tries to find match in the middle
- Instead of 2^{2k} , is now 2^{k+1} time
- Tradeoff between time and space: Can exhaustively select last s bits of each so that less storage space at 2^{k-s+1} but number of operations increases to 2^{s+k+1}
- 3DES: Use triple encryption with 2 keys to make it harder to break, e.g. $E_{k1}(E_{k2}(E_{k1}(x)))$ or $E_{k1}(D_{k2}(E_{k1}(x)))$

Padding Oracle

- An oracle is a query answer system
- If length of plaintext is not a multiple of block size, remaining bytes need to be padded, but receiver also needs to know how much padding was used
- PKCS#7 is a padding standard where padding of x bytes is done with x copies of x , if block is full an extra block of the block size is added
- The padding oracle knows the key and outputs yes if the ciphertext is in the correct padding format
- AES CBC is not secure against padding oracle attack
 - Attacker knows IV, C, and padding format; padded with 3 bytes in this case
 - Attacker calculates iv' from iv by xoring the last 4 bytes where t is set to some value i.e. $0000t777$ to force the padding to flip to 04
 - Attacker feeds padding oracle with (iv', c)

- If oracle outputs YES, then $x_5 \oplus t$ must be 04, and x_5 is $04 \oplus t$
- If not, try with another t value
- Can be extended to find all plaintexts, only information needed is initial padding length which is easy to determine
- Find by flipping from left to right and checking padding
- Solution is to deny access to such an oracle, or avoid padding in the first place if possible
- CTR mode is also vulnerable to padding oracle attack

Cryptography Pitfalls

- Wrong choice of IV: e.g. Deriving IV from filename, which can be repeated across different files, should not be same for different ciphertexts
- Reusing one time pads
- Predictable Secret Generation: Using a fixed seed
- Designing your own cipher or modifying an existing scheme; use well-accepted algorithms and libraries
- Reliance on Obscurity: Kerckhoff's principle says a system should be secure even if everything about the system other than secret key is known
- Security through obscurity is achieving security by hiding system design, but once exposed becomes useless

Authentication (Password)

- Authentication: Assuring that communicating entity or origin of information is the one it claims to be, implies integrity
- Entity Authentication: For connection-oriented communication, verifying authenticity of entities involved
- Data-Origin Authentication: For connectionless communication, verifying origin of a piece of information
- Credential: Information bound to owner which is required for authentication, is authentic with owner can convince verifier it has access to it

Passwords

Password System

- Bootstrapping: User and server establish common password, and server keeps a password file recording the users identity and corresponding password
- Authentication: Server authenticates an entity by ensuring it gives the correct password
- Replay attack: Sniffed information can be replayed to impersonate a user
- Password Reset: User can reset password if necessary, but authentication has to be done first
- Methods include recovery emails and security questions

Password Attacks

- Attacker can intercept password during bootstrapping or exploit usage of default passwords
- Dictionary attack tests all passwords stored in a dictionary and tries all combinations as a form of exhaustive search
- Online dictionary attack: Attacker must interact with authentication system while searching (29 bits minimum, 49 bits recommended)
- Offline dictionary attack: Attacker obtains some information about password, then carries out search using it without interacting with system through methods such as testing hashes (128 bits minimum)
- Social engineering can be carried out to construct a dictionary
- Passwords can be stolen through sniffing in various forms such as key loggers or viruses
- Phishing attacks are social engineering attacks where the goal is to trick the user to visit a spoofed website and key in their login information
- Spear phishing is target to a particular small group of users
- Cache usage on platforms like a browser can also leak passwords

- If the system administrators account is compromised, password files could also be stolen
- Prevention can be done through education and blacklisting phishing sites and using strong passwords
- Password Entropy: Measure of strength of password as a measurement of randomness
 - $-\sum_{i=1}^N p_i \log_2 p_i$, maximised when probability is uniform
- Systems can also have login timeouts or check password strength or require regular password changes, and should also have protected password files using hashing and salting
- Passwords generated by human, secret keys generated by machines; key derivation function (KDF) uses password as source for crypto secret key e.g. SHA3

Biometrics

- Uses physical characteristics of a person for authentication
- During enrollment, a template of a user's biometric data is stored, for verification using a matching algorithm
- Biometric data acts as password, but has a probability of error
- Threshold adjusted for matching algorithm
- Can be spoofed, some systems include liveness detection through methods such as temperature sensors

n -factor Authentication

- Uses more than one authentication factor
- An example is a password and SMS verification or OTP token
- OTP token shares secret with server
- Time Based: Based on shared secret and current time interval, a password is generated
- Sequence Based: An event triggers change of password

Authenticity (Data Origin: MAC & Signature)

Crypto Primitive: PKC (Public Key Crypto)

- Asymmetric key scheme uses different keys for encryption and decryption
- Encryption is done with public key while decryption is done with private key
- Only holder of private key can decipher ciphertext
- Must be difficult to get private key from public key
- Only requires a broadcast channel, does not need a secure channel for sharing of symmetric keys, also uses less keys in total
- Authentication is an extremely important part of PKC
- Many PKC schemes represent data as integers to which arithmetic operations are applied

RSA

- User chooses large prime p, q and computes $n = pq, \varphi(n) = (p - 1)(q - 1)$
- User randomly chooses an encryption exponent e with $\gcd(e, (p - 1)(q - 1)) = 1$
- User finds decryption exponent d where $de \bmod (p - 1)(q - 1) = 1$
- (n, e) is the public key while (n, d) is the private key
- $c = m^e \bmod n$ while $m = c^d \bmod n$
- $D(E(m)) = m, m^{ed} \bmod n = m$ since $m^{de} \bmod n = m \bmod n = m$
- e and d can have their roles swapped so everyone can decrypt but only owner can encrypt
- Common public key is 65537 which is small enough
- Difficulty comes from factorising the large value n
- Post-Quantum Cryptography: PKC that are secure against quantum computers, unlike RSA which will be broken
- IVs needed to get different ciphertexts from the same plaintext at different times

- Not known whether getting plaintext from ciphertext and public key is as difficult as factorisation
- Textbook RSA has to be modified as attacks can use the homomorphic property to extract one bit of information
- 128 bit AES and 3072 bit RSA have equivalent key strength, but RSA is significantly slower
- PKC is useful for authentication
- Typically encrypt AES key with RSA, while using AES to encrypt file instead, send concatenated ciphertext
- Other schemes: Discrete log-based PKC such as ElGamal or Paillier Encryption or Lattice-based PKCs

Data Authenticity (Digest), Unkeyed

- A hash is a function that takes an arbitrary large message as input and outputs a fixed size digest
- Collision-Resistant: Difficult to find different m_1, m_2 such that $h(m_1) = h(m_2)$
- 2nd Preimage: Given m_1 , find m_2 such that $h(m_1) = h(m_2)$
- Preimage: Given y , find m such that $h(m) = y$ (one-way)
- Popular hash functions are the SHA or MD family
- SHA1 and MD5 collisions have been found, use SHA256 or SHA3 instead
- Application: Verifying authenticity of file download, attacker tries to find second preimage that is malicious

Data Origin Authenticity (MAC), Keyed

- A keyed-hash takes an arbitrarily large message and secret key as input and outputs a fixed size message authentication code
- Even after seeing multiple valid pairs of messages and MAC, it is still difficult for an attacker to forge an MAC
- CBC-MAC: Start with IV 0 and repeatedly xor with x_i followed by throwing it in e_k
- Application: When there is no secure channel to deliver digest
- Typically MAC is appended to a file and stored together as authentication tag

Data Origin Authenticity (Signature), Asymmetric Key

- The public key version of MAC is a signature
- Owner uses private key to generate signature which public can verify using public key
- Easier to manage than MAC
- Non-repudiation: Assurance that someone cannot deny previous actions
- Popular schemes use RSA for signing and verifying
- Signature schemes consist of unkeyed hash followed by signing and verifying algorithm
- With RSA, $s = RSA_E(key^-, H(x))$, (X, s) can be verified by checking $H(X) = RSA_D(key^+, s)$
- Some signature schemes do not use encryption in their scheme

Attacks and Pitfalls

- Birthday Attacks: Repeatedly generate messages until probability of finding collision is high
- Suppose we have $M = 1.17 \cdot 2^{64}$ messages and each has a value from $T = 2^{128}$, the probability of collision is $1 - e^{(-\frac{M^2}{2T})}$, basically half the bits, $M > 1.17\sqrt{T}$ gives $P > 0.5$
- For a set of k distinct elments of n bits, select m of them and put them in another set. The probability of non-empty intersection is $> 1 - 2.7^{-km2^{-n}}$
- Using encryption for authentication: Encryption provides confidentiality, but not integrity or authenticity

PKI (Public Key Infrastructure) & Channel Security

Public Key Distribution

- Both public keys and symmetric keys require a secure channel to distribute keys, but it is easier to securely broadcast public keys that do not require both entities to interact
- Public Announcement: Owner broadcasts public key, but not standardised, and there is no systematic way to search and verify them
- Publicly Available Directory: Directory server which supports lookup, but server cannot verify that information is authentic

Public Key Infrastructure

- Standardised system to distribute public keys
- Centered around Certificate and Chain of trust of Certificate Authority (CA)

Certificate Authority

- A CA issues certificates
- Trusted authority that manages directory of public keys
- Has its own public and private key pair, which still needs a secure channel to distribute
- Most OSES and browsers have a few pre-loaded CA's public keys (root CAs)
- Without certificate: User has to take other user's public key and compare it with result of query with CA
- With certificate: User can check certificate and verify signature in certificate belongs to CA, does not need to query CA
- A certificate contains at least a name, public key of owner, time window of validity, and signature of CA
- Standardisation Bodies specify formats for certificates
- A self-signed certificate is signed by its owner and is to be verified using public key listed in certificate
- Convenient in manual installation of public key

Trust Relationship

- CA also has to verify that information is correct before issuing certificate, could be costly
- If user does not have public key of CA of a certificate, needs to obtain it from other CAs

Revocation

- Certificates can be revoked for various reasons such as private key being compromised, CA being compromised, or entity closing / leaving
- Verifier needs to check validity of certificate even if it is not expired
- Certificate Revocation List: CA periodically signs and publishes list of revocations
- Online Certificate Status Protocol: OCSP responder validates a certificate in question
- Either way, some online check has to be done, maybe periodically

Limitations of PKI

- Implementation bugs can lead to vulnerabilities, such as browsers truncating after null characters
- Malicious CAs can forge certificates
- Social engineering exploits confusing naming to trick verifiers into using a wrong name such as typosquatting or sub-domains

Authentication

- Attack Model: Sniff-then-impersonate
- Weak Authentication: Passwords can be replayed for impersonation
- Challenge-Response: Verifier issues a random challenge which entity has to respond to correctly, ensuring freshness using cryptographic nonce

- Unilateral Authentication using PKC: Public key version using signatures using signature of random message and certificate
- Authenticated key exchange: First phase produces a session key, which is then used for protecting communication with encryption and MAC

Key-Exchange

- PKC-Based:
 - Alice generate private and public key pair
 - Alice sends public key to Bob
 - Bob chooses a session key, encrypts it, and sends it to Alice
 - Alice decrypts it to obtain the session key
 - Attacker cannot get session key
- Diffie-Hellman Key-Exchange:
 - Agree on generator g and large prime p
 - Alice randomly chooses a and transmits $x = g^a \bmod p$
 - Bob randomly chooses b and transmits $y = g^b \bmod p$
 - Alice computes $k = y^a \bmod p$
 - Bob computes $k = x^b \bmod p$
 - Assumption: It is computationally infeasible to find $k = g^{ab} \bmod p$ given g, p, x, y

Authenticated Key-Exchange

- Basic key exchange can be attacked by an impersonator in the middle
- Can be done using key-exchange by signing all communication with private keys
- Authenticated key-exchange using DH: Station to Station Protocol
- To authenticate Bob, Bob signs y to obtain a signature which Alice will verify after transmission using Bob's keys
- Can also be done with symmetric keys and passwords

Secure Communication Channel

- Public channel is not secure with presence of attackers
- TLS/SSL: Use long-term keys to carry out authenticated key-exchange (handshake) so that they both have a session key
- Subsequent communication is protected by this session key
- Alice obtains Bob's public key from Bob's certificate
- Alice and Bob carry out unilateral authenticated key exchange with Bob's keys, obtaining session keys t for MAC and k for symmetric key encryption such as AES
- Subsequent interactions use t, k , and a sequence number with information sent being $E_k(i|m) || MAC_t(E_k(i|m))$ (encrypt-then-MAC)
- Or in GCM, send $E_k(i|m)$
- SSL is the predecessor of TLS, HTTPS is built on top of TLS
- Forward Secrecy: Eavesdropper cannot recover plaintext from authenticated key exchange, messages protected by sequence key, and master key

Network Security

Computer Networking

- Collection of interconnected devices
- Uses packet switching where messages are broken down into packets and routed via switches and routers
- Decide how to route messages using intermediate nodes, might need multiple hops
- Network is organised into layers
- Transport + IP Layer often treated as one, address is an IP address and port
- Man-In-The-Middle can see and modify data unit in a specific layer

Name Resolution and Attacks

- DNS (Domain Name System): Resolves domain names into IP addresses
- Client queries DNS server who responds with a query ID and response

- DNS Spoofing: Attacker replies DNS query with another malicious IP address
- DNS server could be single point of failure
- ARP (Address Resolution Protocol): Resolves IP address to MAC address
- Switches connects ports based on MAC addresses
- ARP Poisoning: Attacker inserts false entries into ARP tables so that packets are mistakenly sent to it

Denial of Service Attack

- Attack on availability which disrupts normal functionality of a server or service
- Send large number of http requests to a server, or large number of DNS requests to DNS server
- When carried out by a large number of attackers, known as Distributed DOS
- Reflection Attack: DOS where attackers send requests to intermediate nodes, in turn sending overwhelming traffic to victim
- Amplification Attack: Intermediate nodes response is significantly larger than attacker's request, e.g. using DNS or NTP over UDP
- ICMP/Smurf flood: Attacker sends ICMP ping with source IP of victim, causing all entities who receive broadcast to reply
- Broadcast is disabled by default to prevent this

Securing Layers

- SSL/TLS sits on transport layer to protect data with encryption and MAC
- IPSec protects IP layer by securing network connections
- WPA2 protects between Link and Physical layer; data traveling between wireless device and access point
- VPN enables remote user to securely connect to a private network, establishing a tunnel
- Typically TLS/SSL + WPA2, IPSec is expensive and difficult

Firewalls

- Divide network into segments and deny unnecessary access
- Principle of Least Privilege: Control access to the network
- Compartmentalisation: Confining information within compartments
- A firewall controls what traffic is allowed to enter or leave the network (ingress and egress filtering)
- DMZ: Demilitarised zone; a sub-network exposing organisations external service to the internet
- Typically has front end and back end firewall around DMZ
- Firewalls perform packet filtering which inspects every packet, typically TCP/IP header
- Deep Packet Inspection: Inspecting payload as well, not just headers
- Example rules are checking source IP with whitelists or blacklists
- Rules are defined by matching conditions and actions
- Types: Packet filters, Stateful inspection, Proxy
- Intrusion Detection System: Set of sensors which gather data, to be analysed for intrusion
- Attack signature detection: Attack has specific well defined signature
- Anomaly detection: Attempt to detect abnormal patterns
- Behaviour based: A type of anomaly detection focusing on human behaviour

Web Security

- Complicated due to browsers wide range of support and dealing with sensitive information
- Threat Model 1: Attackers are other end systems
- Threat Model 2: Attackers are MITM
- Vulnerability in Secure Communication Channel: Assumption that system is designed and implemented correctly
- Misleading user with domain name: No clear distinction between hostname and path, can be spoofed using characters that look like /

- Address bar spoofing: In a poorly designed browser, attackers could spoof address bars
- Cookies can have a same origin policy which means it is only sent back to the server that set it
- Same origin policies have confusing definitions which might not be the same between browsers
- Cross Site Scripting: Injecting code in a URL that gets run for example through a content render, allowing attacker to deface webpage or steal cookies and make requests
- Can be reflection (non-persistent) or stored XSS, for example in a forum page, server should carry out input validation
- Cross Site Request Forgery: Exploits server's trust of client by sending a modified request which benefits attacker, can be prevented with authentication
- Drive by Download: Download happening without user consent
- Web Bug: Tiny object embedded in webpage to track user behaviour
- Clickjacking: User tricked into clicking something different from what they actually are
- CAPTCHA: Challenge response test to verify a user is human
- Click Fraud: Malicious actors click on ads to generate fake revenue or drain a competitors budget

Call Stack

- Code is treated as data and both are stored in the same memory
- Program counter is a register that stores address of next instruction
- After an instruction is executed, processor fetches another instruction from memory, and PC is incremented
- Direct branch: PC replaced by constant value
- Indirect branch: PC replace by value fetched from memory

Control Flow Integrity

- Attacker can compromise execution by modifying code in memory or modifying the address for indirect branching
- Can overwrite executing code with malicious code
- Or overwrite a piece of control flow information for either a direct or indirect jump (stack smashing)

Stack

- Call stack stores information of function calls in stack frames
- Stack pointer stores location of the top of the stack
- Call stack keeps track of parameters passed, control flow information such as return address, local variables of functions, pointer to previous stack frame
- Stack Smash: Attacker can modify stack with a buffer overflow

Secure Programming

Unsafe Function e.g. printf

- Uses a format string and parameters
- When format specifier is found in format string, program fetches value of parameter and displays it, even if a second parameter is not given
- By calling printf with a user specified string, an attacker can inject format specifiers to extract information or cause a crash or modify memory content

Data Representation

- Inconsistencies in data representation can cause vulnerabilities
- C strings are represented with null characters marking the end
- A system using both null and non-null definitions might get confused
- Consider a browser which verifies certificates with non-null termination and displays name with null termination
- Attacker can spoof a certificate which has name displayed correctly because an extra null character is inserted

- Do not trust user input, use canonical representations

Buffer Overflow

- C and C++ do not check array bounds at runtime
- A buffer overflow is when data is written beyond a buffer's boundary
- strcpy is an unsafe function because it does not do bounds checking
- Stack smashing is a buffer overflow targeting the call stack

Integer Overflow

- Incrementing beyond maximum value usually loops over to its minimum value
- Some invariants might not hold and can lead to vulnerabilities

Code Injection

- Scripting languages automate execution, interpreted rather than compiled
- Many scripting languages allow scripts to be stored data
- The most well known is an SQL injection, where malicious users can write SQL inside an input field which causes it to be executed

Undocumented Access Point / Easter Egg

- Developers might leave backdoors for debugging purposes
- These might accidentally be left in the production build

Race Condition

- Occurs when multiple processes access a piece of shared data in such a way that outcome depends on order of access
- Time of check time of use race condition is when another process is completed between these two time points
- Example: First program does verification before access, and second program modifies resource being accessed between these two time points

Defense and Preventive Measures

- Input validation, Filtering, Parameterised Queries: Perform input validation whenever an input is obtained from user with a whitelist or blacklist
- Use safe functions rather than unsafe versions which have vulnerabilities
- Bound Checks: Before accessing a location, ensure that it is within bounds
- Type Safety: Ensure that arguments to an operation are always of the correct type
- Canaries: Secrets inserted at selected memory locations during runtime to ensure that values are not being modified
- Memory Randomisation: Address space layout randomisation makes it harder for attacker to know location of data and code for modification
- Code Inspection: Manual or automated checking which checks whether input to critical functions could be affected by user inputs
- Testing: White-box (access to source code), black-box (no access to source code), grey-box testing (combination)
- Fuzzing: Send malformed inputs to discover vulnerabilities
- Principle of Least Privilege
- Patching: Quickly applying patches to fix vulnerabilities when they are found

Access Control

Access Control Model

- Restrict operations on objects by subjects
- Security Parameter: Boundary set up so that malicious activities outside it will not affect resources within it
- Design uses Principle of Least Privilege, Compartmentalisation, Defense in depth, and Segregation of duty
- Defense in Depth: Multiple layers to prevent single point of failure
- Segregation of Duty: Assign different components to different peoples
- Principal / Subject wants to access an object with some operation

- Reference monitor either grants or denies the access
- Principals are human users while subjects are entities acting on behalf of them
- Some accesses are observation, alteration, or actions
- Every object has an owner, access rights decided by owner (discretionary access control) or system-wide policies (mandatory access control)

Access Control Matrix

- How to specify access rights of a particular principal to a particular object
- Use access control matrix to show; one axis principals and the other objects
- ACL: Access control list rather than matrix, for each file
- Capabilities: Subject has a list of capabilities, each for a specific object
- Capability: Unforgeable token that gives possessor certain rights to an object

Intermediate Control

- Specifying every single entry in the access control matrix is tedious
- Grouping subjects and objects and defining access rights on the group simplifies it and is known as intermediate control
- Unix has permissions for owner, group, and world
- Grouping determined by role of subject; collection of procedures
- Privilege is often called protection rings
- A subject with a lower number has higher privilege
- Bell-LaPadula Model for data confidentiality: No read up or write down, prevents lower level from getting information from higher level, and malicious insider from passing information to lower levels
- Biba Model for process integrity: No write up or read down, prevents malicious subjects from poisoning upper level data, and prevents subject from reading data poisoned by lower level subjects

Unix File System

- Objects include files, directories, memory devices, and I/O devices
- File System Permission: Grouped into RWX, for OGW
- File Permission, Links Count, Owner ID, Group ID, File Size, Data and time of last modification, Filename
- Principals are user identities and group identities, with their information stored in password file which is world readable
- Subjects are processes, each with a PID
- Unix has 2 protection rings: superuser which has no security checks, and user
- On access, check if user is owner and owner permission bits, then if user group owns file, check group permission bits, if not check the world permission bits

Controlled Invocation & Privilege Escalation

- Controlled Invocation: Allow user to access a file temporarily
- System provides a predefined set of applications with access to a file
- Application granted elevated privilege to freely access a file and any user can invoke it
- Need to ensure that application only performs intended operation
- Can view predefined application as bridge between user and sensitive data
- Privilege Escalation Attack: Attacker tricks bridge to perform illegal operations

Controlled Invocation in UNIX

- A process is a subject with a PID, which can be created by executing a file or forking an existing process
- A process is associated with a Real UID and Effective UID
- Real UID is inherited from user who invokes the process
- If Set User ID is disabled, process effective UID is same as real UID
- If enabled, process effective UID is inherited from UID of executable's owner
- When process wants to access a file, effective UID of process is treated as subject and checked against file permission to determine whether it has access
- SUID must be enabled in file permissions
- Process is temporarily elevated to superuser so it can access the sensitive data